

ADCS Simulation and Control System Documentation

Sebastian Benner

Contents

Contents	I
1. ADCS Simulation and Software Architecture	2
1.1. Overview	2
1.2. Simulation Environment	2
1.2.1. Sensors	3
1.2.2. Actuators	3
1.3. Simulation Configuration	3
1.3.1. Spacecraft Properties	3
1.3.2. Initial Conditions	4
1.3.3. Sensor Configuration	4
IMU	4
Star Tracker	4
1.3.4. Guidance and Targeting	4
1.3.5. Control Configuration	5
1.3.6. Simulation Timing	5
1.3.7. Visualization and Output	5
1.3.8. Additional Features	5
1.3.9. Configuration Assembly	5
1.4. Flight Software Architecture	6
1.4.1. Simulation Flight Software	6
1.4.2. Hardware Flight Software	6
1.4.3. Key Distinction	6
1.5. Coordinate Frames and Conventions	6
1.5.1. Quaternion Representation	6
1.5.2. Reference Frames	7
2. State Estimation	8
2.1. Overview	8
2.2. Asynchronous Sensor Fusion	8
2.3. State Definition	8
2.4. Prediction	9
2.5. Correction	9
2.6. Implementation Details	9
3. Guidance	10
3.1. Overview	10
3.2. Guidance Modes	10
3.3. Target Orientation Definition	10
3.4. Overpass Determination	10
4. Control System	12
4.1. State-Space Model	12
4.2. Control Law	12

4.3. LQR Control Design	12
4.3.1. Weighting Matrices	13
4.4. Gain Design	13
4.5. Variable Gain Control	13
4.6. Integrator Augmentation	14
4.7. Feedforward Terms	14
4.8. Actuator Allocation	14
4.9. Assumptions and Limitations	15
4.10. Known Issues	15
5. Simulation vs. Flight Software Implementation	16
5.1. Purpose	16
5.2. Algorithmic Consistency	16
5.3. Key Implementation Differences	16
5.3.1. Sensor Inputs	16
5.3.2. Actuator Behavior	17
5.3.3. Timing and Execution	17
5.3.4. Numerical and Software Environment	18
5.4. Practical Considerations for Equivalence	18
5.5. Summary	18
A. Reference Frames and Coordinate Systems	19

1. ADCS Simulation and Software Architecture

1.1. Overview

The Attitude Determination and Control System (ADCS) simulation environment is designed to support the development, validation, and verification of spacecraft attitude estimation and control algorithms.

The system consists of three primary components:

- A Basilisk-based spacecraft simulation environment
- A simulation flight software (FSW) implementation
- A hardware flight software implementation

All core ADCS algorithms, including estimation, guidance, and control, are shared between simulation and hardware implementations.

The simulation is executed from the working directory in the command terminal via:

```
python ADCS_Simulation_Basilisk.py
```

1.2. Simulation Environment

The simulation is implemented using the Basilisk framework and is structured into two primary processes:

- **Dynamics Process**
 - Rigid body spacecraft dynamics
 - Orbital propagation
 - Environmental models (gravity, magnetic field)
- **Flight Software Process**
 - Sensor data processing
 - State estimation
 - Guidance computation
 - Control execution

1.2.1. Sensors

The following sensors are simulated:

- Star tracker (attitude measurement)
- IMU (angular velocity measurement)
- Magnetometer (magnetic field vector)

Sensor models include stochastic noise and bias effects to approximate real hardware behavior.

1.2.2. Actuators

- Reaction wheels (primary control authority)
- Magnetorquers (detumbling and auxiliary control)

1.3. Simulation Configuration

The simulation is configured via a centralized parameter dictionary constructed in the main execution script. This configuration defines spacecraft properties, sensor characteristics, control modes, and simulation timing.

1.3.1. Spacecraft Properties

The spacecraft model is selected at runtime. Each model defines:

- Inertia tensor J
- Mass
- Visualization model

The inertia tensor is defined as:

$$J = \begin{bmatrix} J_{xx} & J_{xy} & J_{xz} \\ J_{yx} & J_{yy} & J_{yz} \\ J_{zx} & J_{zy} & J_{zz} \end{bmatrix}$$

The simulation supports multiple spacecraft configurations (e.g., SENTINEL, OreSat), each with distinct inertial properties.

The spacecraft mass is specified as a scalar parameter and used within the Basilisk dynamics model.

1.3.2. Initial Conditions

The initial attitude is defined using an axis-angle representation:

- Initial rotation axis
- Initial rotation angle

The initial angular velocity is specified in RPM and converted to radians per second:

$$\omega_{init} = \omega_{RPM} \cdot \frac{2\pi}{60}$$

An additional commanded rotation can be applied to define the initial orientation of the satellite.

1.3.3. Sensor Configuration

Sensor models include both stochastic noise and bias terms.

IMU

- σ_{gyro} : white noise standard deviation
- σ_{bias} : bias random walk parameter
- P_{b0} : initial gyro bias uncertainty

Star Tracker

- σ_{ST} : measurement noise
- $P_{ST,0}$: initial covariance
- Update rate (asynchronous measurement availability)

These parameters are used both in the Basilisk sensor models and the MEKF covariance definitions.

1.3.4. Guidance and Targeting

Guidance behavior is controlled via:

- `guidance_mode`: TARGET, NADIR, MAX_DRAG, MIN_DRAG, or None
- Target coordinates (latitude, longitude, altitude)
- `activate_on_overpass`: enables ground-pass-triggered activation

Target positions are converted to ECEF coordinates for guidance computations.

1.3.5. Control Configuration

The control system is defined via:

- `control_mode`: DETUMBLE, RW_POINTING, MTB_POINTING, THERMAL modes
- `pointing_reference`: defines boresight-reference or pointing-reference axis of the spacecraft
- `use_variable_gain`: enables gain scheduling for fine pointing

Control modes determine which actuators are active and how control laws are applied.

1.3.6. Simulation Timing

Simulation timing is dependent on control mode:

- `sim_time`: total simulation duration
- `dynamics_update_time`: integration timestep
- `fsw_update_time`: controller update rate

Different modes use different time scales:

- Reaction wheel control: high-frequency updates (e.g., 0.01–0.1 s)
- Magnetorquer control: lower-frequency updates (e.g., 5–20 s)
- Orbit visualization: coarse updates for long-duration simulation

1.3.7. Visualization and Output

The simulation supports optional visualization and data export:

- 3D model selection and scaling
- Plot saving (PDF, PNG)
- Output directory configuration

1.3.8. Additional Features

- `use_filter`: enables MEKF-based state estimation with simulated sensor noise
- `use_skyfield`: enables the usage of Skyfield Ephemeris data for guidance functions and simulation
- `error_time_check`: defines time window to be excluded from performance metric evaluation to avoid transient states from skewing data

1.3.9. Configuration Assembly

All parameters are collected into a single dictionary and passed to the simulation entry point:

```
config = {...}
sim_main(config)
```

This design enables centralized configuration and consistent parameter propagation across all modules.

1.4. Flight Software Architecture

1.4.1. Simulation Flight Software

The simulation flight software interfaces with the Basilisk messaging system and operates within the simulation time framework.

1.4.2. Hardware Flight Software

The hardware implementation interfaces with real sensors and actuators via a service-based architecture.

1.4.3. Key Distinction

The primary difference between the two implementations lies in:

- Data acquisition and communication interfaces

All estimation, guidance, and control algorithms are identical between simulation and hardware implementations.

1.5. Coordinate Frames and Conventions

1.5.1. Quaternion Representation

Quaternions are represented in scalar-last form:

$$q = [q_x, q_y, q_z, q_s]^T = \begin{bmatrix} q_v \\ q_s \end{bmatrix}$$

where $q_v = [q_x, q_y, q_z]^T$ is the vector portion and q_s is the scalar portion.

A quaternion representing a rotation of angle θ about a unit axis $\hat{e}$ is given by:

$$q = \begin{bmatrix} \hat{e} \sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}$$

Quaternion multiplication for this project follows the Shuster convention.

The attitude error quaternion is defined as the relative rotation between the current and desired spacecraft attitudes. In the control formulation, only the vector portion of the error quaternion is used directly in the state vector and control law. For small attitude errors,

$$q_{e,v} \approx \frac{1}{2}\delta\theta$$

which motivates the linearized state-space formulation used for controller design.

1.5.2. Reference Frames

- Body frame (B)
- Inertial frame (ECI)
- Earth-fixed frame (ECEF)

All angular velocities and control torques are expressed in the body frame. Appendix elaborates on reference frame nomenclature.

2. State Estimation

2.1. Overview

Attitude estimation is performed using a Multiplicative Extended Kalman Filter (MEKF).

2.2. Asynchronous Sensor Fusion

The state estimation architecture is designed to operate with asynchronous sensor updates. This is required due to the differing update rates of the onboard sensors.

The IMU provides angular velocity measurements at a high rate, while the star tracker provides attitude measurements at a significantly lower rate (on the order of 2 Hz). Additionally, due to uncertainty in the CAN bus communication protocols, exact timings of these sensor updates cannot be guaranteed.

The Multiplicative Extended Kalman Filter (MEKF) is structured to accommodate this disparity in measurement availability:

- IMU measurements are used continuously in the propagation step
- Star tracker measurements are incorporated only when available for correction

Between measurement updates, the filter performs state propagation using a zero-order hold assumption on angular velocity. When higher-rate IMU data is available, midpoint integration is used to improve propagation accuracy.

This asynchronous design ensures that the estimator remains stable and accurate despite the differing sensor update rates.

2.3. State Definition

$$x = \begin{bmatrix} \delta\theta \\ b \end{bmatrix}$$

where $\delta\theta$ represents the small-angle attitude error and b represents gyro bias.

2.4. Prediction

- Quaternion propagation via exponential map
- Covariance propagation using linearized dynamics

2.5. Correction

- Innovation derived from quaternion error
- Joseph-form covariance update for numerical stability

2.6. Implementation Details

- Hemisphere enforcement is required for stability
- Midpoint integration is used when IMU updates are available
- Zero-order hold is used otherwise

3. Guidance

3.1. Overview

Guidance defines the desired spacecraft orientation independently of control.

3.2. Guidance Modes

Supported modes include:

- Target tracking
- Nadir pointing
- Drag optimization (maximum or minimum drag orientation)

3.3. Target Orientation Definition

Target quaternions are constructed from geometric relationships between spacecraft position, velocity, and Earth-relative target vectors.

3.4. Overpass Determination

In addition to real-time guidance computation, the system includes functionality to determine ground station overpass windows.

These overpass functions are used to determine when the ADCS system should be active or inactive. They are not intended to be executed within the ADCS flight software itself, but rather externally as part of mission planning or higher-level system logic. In the simulation environment, these functions are invoked directly for testing and validation purposes.

The overpass determination is performed using Keplerian orbit propagation. While computationally efficient, Kepler propagation introduces increasing error over longer time horizons due to unmodeled perturbations.

To account for this limitation, overpass prediction is performed in two stages:

- A coarse search is performed up to 24 hours in advance to identify approximate overpass windows

- A refined search is performed once the spacecraft is within 6 hours of the predicted overpass window to improve accuracy

This two-stage approach balances computational efficiency with sufficient accuracy for operational use.

The resulting overpass windows are used to determine when the ADCS system should transition between active pointing modes and inactive or low-power states.

4. Control System

4.1. State-Space Model

$$\dot{x} = \begin{bmatrix} 0 & \frac{1}{2}I \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ J^{-1} \end{bmatrix} u$$

4.2. Control Law

The controller state vector is defined as

$$x = \begin{bmatrix} q_{e,v} \\ \omega \end{bmatrix}$$

where $q_{e,v}$ is the vector portion of the attitude error quaternion and ω is the body-frame angular velocity error. This is used in the resulting control law:

$$\tau = u = -Kx$$

Where u is the desired 3-axis body torques of the satellite, and K is the gain matrix derived from the LQR control design.

4.3. LQR Control Design

The control system is designed using a Linear Quadratic Regulator (LQR) formulation.

The gain matrix K is computed by solving the discrete-time algebraic Riccati equation, following discretization of the continuous-time system using a zero-order hold assumption.

The LQR design is based on qualitative tuning parameters, including:

- Maximum attitude error
- Maximum angular rate
- Maximum actuator torque

These values are not strict physical limits, but rather representative scaling values used to define the relative importance of each state and input. As a result, controller performance is governed by

the relative weighting between attitude error, angular velocity, and control effort, rather than the absolute numerical values chosen.

The relationship between these qualitative tuning parameters and the resulting controller gains is established through the construction of the weighting matrices.

4.3.1. Weighting Matrices

The cost function minimized by the LQR controller is of the form:

$$J = \sum (x^T Q x + u^T R u)$$

The matrices Q and R define the relative weighting of the state error and control effort in the cost function.

- Q penalizes deviations in the state vector, including attitude error and angular velocity
- R penalizes control input effort (actuator torque)

The weighting matrices are constructed using the previously defined qualitative parameters for maximum expected attitude error, angular rate, and actuator torque. These values are used as scaling factors to normalize each term in the cost function.

Specifically, each state and input is weighted inversely proportional to the square of its corresponding characteristic value. This ensures that states with smaller acceptable magnitudes (such as attitude error) are penalized more heavily than those with larger acceptable magnitudes.

As with the overall LQR design, the absolute values used in constructing Q and R are not physically prescriptive. Instead, the resulting controller behavior is determined by the relative weighting between the different state components and control effort. Adjusting these parameters effectively shifts the balance between responsiveness, control authority, and smoothness of the system response.

4.4. Gain Design

- Discretization via zero-order hold
- Gain computation done using the discrete Riccati equation

4.5. Variable Gain Control

An optional variable gain control scheme is implemented to improve fine-pointing performance.

This approach utilizes two separate gain matrices:

- A nominal gain matrix for coarse pointing

- A secondary gain matrix with increased weighting on quaternion error for fine pointing near the target

When the spacecraft approaches the target attitude, the controller transitions from the nominal gain matrix to the fine-pointing gain matrix.

Although a smoothing transition is applied between the two gain sets, this switching behavior introduces a small transient response in the actuator commands. In practice, this manifests as a slight jerk in the reaction wheel torques.

Figure 4.1.: Transient response during gain transition (reaction wheel torque)

4.6. Integrator Augmentation

A method for incorporating an integrator term into the controller was implemented to reduce steady-state error.

However, it was found that the baseline controller already achieves extremely small steady-state errors. In practice, the introduction of an integrator provided no meaningful improvement in performance.

Instead, the presence of step changes in the target attitude resulted in increased transient instability when the integrator term was active. These effects outweighed any marginal benefit in steady-state accuracy.

Given that the current steady-state errors are already negligible, and that the real hardware system is expected to exhibit greater limitations due to actuator and sensor imperfections, the integrator-based controller was not retained in the final implementation.

4.7. Feedforward Terms

Feedforward terms are included to:

- Track time-varying reference attitudes
- Compensate for gyroscopic coupling

4.8. Actuator Allocation

Desired body torque is mapped to reaction wheel torques using a wheel configuration matrix.

The matrix $G \in \mathbb{R}^{3 \times n}$ defines the orientation of each reaction wheel in the spacecraft body frame. Each column of G corresponds to the unit vector of a reaction wheel spin axis expressed in body coordinates:

$$G = \begin{bmatrix} | & | & & | \\ g_1 & g_2 & \cdots & g_n \\ | & | & & | \end{bmatrix}$$

where each $g_i \in \mathbb{R}^3$ is a unit vector representing the axis of the i -th reaction wheel.

The relationship between individual wheel torques u_{rw} and the resulting body torque τ is given by:

$$\tau = Gu_{rw}$$

Since the system is overactuated for the OreSat/SENTINEL bus for redundancy (e.g., four wheels for three axes), the inverse mapping is computed using the Moore-Penrose pseudoinverse:

$$u_{rw} = G^\dagger \tau$$

where $G^\dagger$ is the pseudoinverse of the configuration matrix.

This formulation provides a least-squares solution that minimizes control effort while achieving the desired body torque. Care must be taken to ensure consistent sign conventions between the definition of G , the actuator model, and the applied torque commands.

4.9. Assumptions and Limitations

- Rigid body dynamics
- Small-angle approximation in estimation
- Simplified actuator saturation modeling

4.10. Known Issues

- Sensitivity to quaternion sign conventions
- Reaction wheel allocation sign consistency
- Dependence on accurate frame transformations
- Small transient torque discontinuity during variable gain transitions

5. Simulation vs. Flight Software Implementation

5.1. Purpose

The flight software (FSW) and simulation are designed to be algorithmically identical, ensuring that all control, estimation, and guidance behavior observed in simulation directly translates to flight.

Differences between the two implementations arise solely from:

- hardware interfaces
- timing and execution constraints
- real sensor and actuator behavior

No intentional changes are made to control laws, state definitions, or estimation algorithms.

5.2. Algorithmic Consistency

All core ADCS components are shared conceptually between simulation and flight:

- Attitude representation (quaternions, scalar-last convention)
- Error computation (quaternion error definition and hemisphere handling)
- Control law (LQR / PD structure using quaternion vector error and body rates)
- State definitions and units

This ensures:

- identical closed-loop behavior in the absence of disturbances
- direct validation of controller performance in simulation

5.3. Key Implementation Differences

5.3.1. Sensor Inputs

Simulation:

- Perfect or modeled sensor data (e.g., noiseless or parametrically noisy IMU, star tracker)

Flight:

- Real sensor measurements with:
 - noise
 - bias
 - misalignment
 - dropouts and latency

Implication: The estimation pipeline (filters, normalization, validation) becomes critical in flight but is simplified in simulation.

5.3.2. Actuator Behavior

Simulation:

- Ideal torque application or simplified actuator models

Flight:

- Real actuators with:
 - saturation limits
 - rate limits
 - deadbands
 - delays

Implication: Commands generated by the controller may not be realized exactly, requiring robustness to actuator constraints.

5.3.3. Timing and Execution

Simulation:

- Deterministic, fixed-step integration
- Synchronous updates between modules

Flight:

- Discrete-time execution with:
 - task scheduling
 - variable latency
 - asynchronous sensor updates

Implication: The controller operates under sampled-data conditions and must tolerate timing jitter.

5.3.4. Numerical and Software Environment

Simulation:

- High-performance computing environment (desktop CPU/GPU)
- Double precision typically available

Flight:

- Embedded processor constraints:
 - limited compute resources
 - possible reduced precision
 - strict real-time requirements

Implication: Algorithms must remain computationally efficient and numerically stable.

5.4. Practical Considerations for Equivalence

To maintain consistency between simulation and flight:

- Identical mathematical formulations are used for all algorithms
- Frame definitions and conventions are strictly preserved
- Quaternion normalization and hemisphere policies are enforced consistently
- Interfaces are designed so that simulated messages match flight message structures

Where possible, simulation components are structured to mirror flight software interfaces, enabling direct reuse or minimal translation.

5.5. Summary

The simulation environment serves as a high-fidelity representation of the flight system, with the same underlying ADCS algorithms.

Differences between simulation and flight are not algorithmic, but stem from:

- real-world sensor and actuator behavior
- timing and execution constraints
- embedded system limitations

As a result, simulation performance is expected to closely match on-orbit behavior, provided these non-ideal effects are properly accounted for.

A. Reference Frames and Coordinate Systems

Reference Points (First Two Letters)

Index	Meaning
B	Body frame origin (hub reference point or geometric center)
C	Spacecraft Center of Mass (CoM)
E	Earth center
S	Sun center
M	Moon center
N	Inertial frame origin
H	Hill frame origin
O	Generic reference origin

Frame Index (Last Letter)

Index	Meaning
N	Inertial frame (J2000 orientation)
B	Body frame
E	Earth-fixed frame (ECEF)
H	Hill / LVLH frame
S	Sun frame

Common Basilisk State Vectors

Variable	Meaning
r_CN_N	Position of spacecraft CoM relative to inertial origin, in inertial frame
v_CN_N	Velocity of spacecraft CoM relative to inertial origin, in inertial frame
r_BN_N	Position of body frame origin relative to inertial origin, in inertial frame
v_BN_N	Velocity of body frame origin relative to inertial origin, in inertial frame
r_CE_N	Position of spacecraft CoM relative to Earth center, in inertial frame
v_CE_N	Velocity of spacecraft CoM relative to Earth center, in inertial frame
r_CE_E	Position of spacecraft CoM relative to Earth center, in Earth-fixed frame
v_CE_E	Velocity of spacecraft CoM relative to Earth center, in Earth-fixed frame